Guia de apresentação — Assistente operacional da FBS Distribuição

Para que serve este guia

Este material explica o projeto em linguagem de apresentação: o problema, a solução, as regras de interpretação, os dados, o aprendizado de máquina, a arquitetura, os resultados e as limitações. A ideia é ajudar a explicar o trabalho com segurança, sem confundir a proposta acadêmica com o que o protótipo já implementa.

1. Resumo em uma frase

O projeto é um chatbot de consultas operacionais para um centro de distribuição fictício. A pessoa escreve uma pergunta no Telegram; o sistema identifica o assunto, extrai identificadores como embarque, placa ou nota fiscal, consulta bancos SQLite com dados sintéticos e devolve uma resposta curta. A interpretação combina regras explícitas e um classificador tradicional de texto.

2. Problema e motivação

Em uma operação logística, dados sobre embarques, agendamentos, pátio, notas fiscais e ocorrências podem estar separados em sistemas diferentes. Uma pergunta simples, como “onde está minha carga?”, pode obrigar alguém da equipe a procurar a mesma operação em várias telas. Isso aumenta o tempo de resposta, o retrabalho e a chance de respostas divergentes.

O projeto investiga se um canal de conversa pode servir como ponto único de consulta. O chatbot não substitui os sistemas operacionais: ele recebe a pergunta e reúne informações guardadas nas fontes simuladas.

Fala sugerida

“A motivação foi reduzir a fragmentação das consultas operacionais. Em vez de o usuário descobrir em qual sistema está cada informação, ele pergunta em linguagem natural e o assistente consulta as bases simuladas.”

3. Objetivos

- Centralizar consultas operacionais em uma interface Telegram.
- Simular três sistemas independentes: TMS, pátio e ocorrências.
- Criar um corpus rotulado com dez intenções.

- Comparar modelos clássicos de classificação de texto usando TF-IDF.
- Combinar o melhor classificador com regras e expressões regulares.
- Demonstrar controle de acesso, auditoria e cálculo ilustrativo de estadia.
- Medir a classificação em validação cruzada e em um conjunto OOD provisório.

OOD significa out-of-distribution: exemplos de teste que procuram diferir das frases de desenvolvimento, incluindo gírias, abreviações, erros e mensagens fora do domínio. Neste projeto, o conjunto é provisório e foi gerado para o protótipo; não deve ser descrito como uma coleta independente feita com usuários reais.

4. O que acontece quando chega uma mensagem

O fluxo simplificado é:

1. O adaptador do Telegram recebe a mensagem e o chat_id da conversa.
2. O motor do projeto verifica o texto e procura entidades: código de embarque, placa e número de NF.
3. Regras de palavras-chave tentam reconhecer saudações, ajuda e assuntos operacionais diretos.
4. Um identificador isolado, como EMB000010, pode indicar a intenção mesmo sem uma frase completa.
5. Se as regras não resolverem a mensagem, o classificador escolhe uma das dez classes e fornece uma confiança calibrada.
6. Se a confiança for menor que 0,55, o motor usa o fluxo de fallback.
7. Para uma consulta operacional, a camada de serviços consulta o SQLite e monta uma resposta em texto simples.
8. O sistema grava uma linha de auditoria, mascarando identificadores no texto persistido.

As mensagens são respondidas em blocos curtos para leitura no celular. O motor de interpretação foi separado do adaptador Telegram e também pode ser chamado pelo CLI.

5. As dez intenções

Intenção • O que o usuário quer • Exemplo
consultar_embarque • Situação e dados consolidados de um embarque • “status do EMB000010”
consultar_agendamento • Data, doca e status do agendamento • “agendamento do EMB000010”
consultar_nota_fiscal • Procurar uma nota fiscal • “nf 123456”
consultar_veiculo_patio • Consultar movimentação de uma placa • “placa ABC1D23 está no pátio?”
consultar_ocorrencia • Consultar ocorrências de um embarque • “ocorrências do EMB000010”
consultar_estadia • Estimar excedente de permanência • “quanto de estadia do EMB000010?”
consultar_transportadora • Consultar uma transportadora e embarques associados • “transportadora Transportadora 01”
saudacao • Cumprimento, agradecimento ou despedida • “oi”, “obrigado”

ajuda • Pedir orientação de uso • “ajuda”

fora_escopo • Mensagem não suportada pelo domínio • “qual a previsão do tempo?”

O conjunto de classes é uma decisão inicial do experimento. Ele pode ser ampliado, mas ao adicionar uma intenção é necessário incluir exemplos rotulados, treinar novamente e atualizar os testes, regras e respostas.

6. Regras determinísticas e extração de entidades

Regras de intenção

O código procura palavras e expressões indicativas. Por exemplo:

- termos como “oi”, “bom dia” e “obrigado” podem levar a saudacao;
- “ajuda” ou “comandos” pode levar a ajuda;
- “ocorrência”, “avaria” ou “atraso” pode levar a consultar_ocorrencia;
- “agendamento”, “agenda” ou “doca” pode levar a consultar_agendamento;
- “estadia” ou “permanência” pode levar a consultar_estadia;
- “nota fiscal”, “danfe” ou “NF” pode levar a consultar_nota_fiscal;
- “placa” ou “veículo” pode levar a consultar_veiculo_patio.

Essas regras são transparentes e rápidas, mas podem ser ambíguas. Como exemplo, “doca” é uma pista para agendamento; se a intenção for outra, pode ser necessário reformular a pergunta.

Expressões regulares

As entidades são extraídas por padrões:

- embarque: EMB seguido de seis dígitos, como EMB000010;
- placa antiga: três letras e quatro dígitos, por exemplo ABC-1234;
- placa Mercosul: três letras, um dígito, uma letra e dois dígitos, por exemplo ABC1D23;
- NF: seis a nove dígitos.

As expressões regulares ajudam a encontrar identificadores previsíveis, mas não descobrem por si só qual consulta o usuário pretende fazer. A combinação de intenção e entidade é que orienta a consulta.

Transportadora e correspondência aproximada

O projeto usa rapidfuzz para comparar texto com nomes de transportadoras e tolerar alguma diferença de digitação. O limiar configurado é 80. Isso é uma aproximação: uma comparação feita contra uma frase inteira pode falhar ou ficar ambígua. A identificação por nome deve ser validada com exemplos reais antes de uma implantação.

7. Classificação de texto

Pré-processamento

Antes da vetorização, o texto é transformado em minúsculas, transliterado para remover acentos e pontuação, e espaços repetidos são normalizados. Assim, variações como “ação” e “acao” ficam mais parecidas para o modelo.

TF-IDF e bigramas

O TF-IDF representa um texto por números associados aos termos. Um termo tende a receber mais peso quando é importante naquele texto e menos peso quando aparece em quase todos os exemplos. A configuração usa unigramas e bigramas (ngram_range=(1, 2)): palavras individuais e pares adjacentes, como “nota fiscal”.

Essa representação é adequada para uma linha de base clássica: é leve, rápida e interpretável em comparação a uma rede neural profunda.

Modelos comparados

1. Multinomial Naive Bayes: modelo probabilístico simples, usado como baseline para texto.
2. LinearSVC calibrado: classificador linear que separa as classes em um espaço de muitas características. A calibração sigmoid transforma os escores em valores utilizáveis como probabilidades aproximadas.
3. Random Forest calibrado: conjunto de árvores de decisão, incluído para comparação com um modelo de ensemble.

Todos os candidatos têm calibração sigmoid no pipeline experimental. A calibração não significa que a probabilidade é uma certeza; ela permite aplicar um limiar de confiança, mas deve ser avaliada com mais dados.

Treino e escolha

O script experimental usa GridSearchCV, StratifiedKFold com cinco partições e F1-macro como critério. A divisão estratificada procura manter a proporção das classes em cada partição. O grid testa configurações limitadas de hiperparâmetros, como C do SVM, alpha do Naive Bayes e número/profundidade das árvores.

O pipeline salvo para execução do bot é o LinearSVC calibrado. Os resultados de validação cruzada orientam a escolha, mas a diferença entre os três valores de CV F1 é pequena. Isso não demonstra que o SVM será sempre superior.

8. Corpus e avaliação

Dados de desenvolvimento

O script gera 2.000 mensagens de desenvolvimento: 200 exemplos para cada uma das dez classes. As frases vêm de templates, sufixos e algumas variações. Textos duplicados são removidos antes da validação cruzada.

Teste OOD

O conjunto OOD atual tem 210 exemplos e inclui gírias, abreviações e exemplos adicionais destinados a testar regras, como embarque, placa e NF isolados. Ele é útil para demonstrar o fluxo, mas ainda é pequeno e sintético. Os exemplos de data/corpus_ood_modelo.csv devem ser substituídos ou complementados por frases inéditas preparadas pelos autores/colegas antes de apresentar os números como uma avaliação final independente.

Métricas explicadas

- Acurácia: proporção total de previsões corretas.
- Precisão: entre as mensagens previstas como uma classe, quantas realmente pertenciam a ela.
- Recall (revocação): entre as mensagens que realmente pertenciam a uma classe, quantas foram encontradas.
- F1: equilíbrio entre precisão e recall.
- F1-macro: calcula o F1 de cada classe e depois faz a média simples. Cada classe tem peso igual, mesmo que a quantidade de exemplos varie.
- Latência: tempo medido para processar uma mensagem na avaliação local; não é uma garantia de tempo de resposta da rede Telegram ou do banco em produção.

Resultados que constam no relatório

Modelo	F1-macro CV (5 folds)	Acurácia OOD	F1-macro OOD
MultinomialNB calibrado	0,932	0,710	0,675
LinearSVC calibrado	0,933	0,695	0,662
Random Forest calibrado	0,934	0,538	0,519

No teste ponta a ponta OOD:

- ML-only: acurácia 0,695 e F1-macro 0,708;
- híbrido (regras + identificadores isolados + ML + limiar): acurácia 0,767 e F1-macro 0,747;
- latência híbrida medida: média de 2,198 ms por mensagem e p95 de 4,268 ms.

O ganho de acurácia reportado para o híbrido é de 7,2 pontos percentuais nesse conjunto. Não é uma garantia de melhoria em produção. O conjunto é pequeno, gerado sinteticamente e inclui exemplos explícitos para as regras; com outros dados, os números podem mudar.

Limiar de confiança

Quando a maior probabilidade prevista pelo classificador é menor que 0,55, a mensagem é encaminhada para fora de escopo. O objetivo é evitar responder com uma intenção pouco confiável. Há um compromisso: aumentar o limiar tende a produzir mais fallbacks de mensagens legítimas; diminuir o limiar pode aceitar mais mensagens que não pertencem ao domínio.

No relatório, com o limiar 0,55, o fallback dentro do escopo foi 22,1% para ML-only e 15,3% para o híbrido; para mensagens fora de escopo, foi 25,0% em ambos. Esses valores dependem da composição daquele conjunto de teste.

9. Dados e bancos simulados

O projeto usa quatro arquivos SQLite:

- data/tms.db: transportadoras, embarques, notas fiscais e agendamentos;
- data/patio.db: placa, embarque e horários de chegada, entrada na doca e saída;
- data/ocorrencias.db: tipo e descrição da ocorrência associada a um embarque;
- data/app.db: usuários autorizados e auditoria.

O embarque é a ligação entre as fontes. A aplicação conecta os bancos e consulta os dados relacionados para compor a resposta. Os dados não são dados de uma operação real.

Usuários e autorização: limitação importante

A tabela usuarios associa um chat_id a um perfil. O código de autorização tem regras para interno, transportadora e motorista. Contudo, a revisão do código atual mostra que as verificações não são aplicadas de forma uniforme a todas as intenções; por exemplo, algumas rotas de consulta de NF, pátio, agendamento e ocorrência não fazem a mesma verificação de escopo que a consulta do embarque. O cadastro de perfis demonstra a ideia, mas não deve ser apresentado como controle de acesso pronto para dados reais. Isso precisa ser corrigido e testado antes de qualquer uso corporativo.

Auditoria e privacidade

O serviço de auditoria mascara padrões de placas e números de NF no texto e inclui uma função para remover registros antigos, com retenção padrão planejada de 90 dias. A rotina de limpeza existe como função, mas não há evidência de que seja executada automaticamente no bot. A auditoria atual também não deve ser descrita como uma implementação completa de conformidade LGPD. Um sistema real precisaria de análise jurídica, política de retenção, controles de acesso, segurança, gestão de incidentes e avaliação dos dados armazenados.

Estadia: definição e limitações

A regra calcula:

horas excedentes = máximo(0, horas desde a chegada − 5)

Quando configurado, o valor é:

valor estimado = horas excedentes × R$ 60,00

O resultado é uma estimativa baseada no horário de chegada e no relógio local do processo. O código atual não usa o horário de saída para calcular uma estadia encerrada; portanto, não é uma apuração financeira/legal. Os horários do banco são sintéticos e podem ficar antigos, gerando valores grandes. O valor de R$ 60/h é uma hipótese configurável do protótipo, não deve ser apresentado como valor legal ou definitivo.

10. Tecnologias e execução

- Python: linguagem principal.
- SQLite: persistência local dos bancos sintéticos.
- pandas: leitura dos corpora tabulares.
- scikit-learn: TF-IDF, classificadores, calibração, validação cruzada e métricas.
- Unidecode: remoção/transliteração de acentos no pré-processamento.
- rapidfuzz: correspondência aproximada de nomes.
- python-telegram-bot: recebimento e envio de mensagens pelo Telegram usando polling.
- joblib: gravação e carregamento do pipeline treinado.
- pytest: testes automatizados.
- matplotlib: matriz de confusão e visualizações do experimento.

Para reproduzir os resultados, a partir da raiz do projeto:

```
.\.venv\Scripts\Activate.ps1
python scripts/seed_db.py
python scripts/generate_corpus.py
python scripts/train.py
python experiments/run_experiments.py
pytest
```

Para iniciar o bot, o arquivo .env precisa conter um token válido em TELEGRAM_BOT_TOKEN. O token é secreto e não deve ser colocado em slides, no relatório, no GitHub ou em mensagens públicas. O bot só atende enquanto o processo de polling estiver em execução.

11. O que já foi testado e o que falta

O estado verificado do protótipo inclui geração dos bancos e corpora, treino do modelo, execução dos experimentos, execução do CLI e uma suíte de dez testes passando. Os testes existentes verificam pré-processamento, algumas entidades, cálculo de estadia, mascaramento, perfis, ambiguidade, expiração da auditoria, classificador, fluxo do motor e formatadores.

Ainda são necessários, para uma avaliação acadêmica mais forte:

1. Um conjunto OOD novo e escrito independentemente pelos autores ou participantes, sem frases derivadas dos templates.
2. Mais exemplos reais/simulados cuidadosamente rotulados, incluindo erros e mensagens ambíguas.
3. Testes de integração para cada intenção e para cada perfil de acesso.
4. Correção e validação das verificações de autorização em todas as consultas.
5. Um fluxo seguro e testado de seleção de transportadora ambígua; a implementação atual ainda precisa validar que a escolha subsequente conclui a consulta.
6. Cálculo de estadia com entrada e saída, fuso horário e regra de arredondamento definidos.
7. Execução automática e documentada da rotina de retenção da auditoria.
8. Avaliação com diferentes sementes/datasets e análise de erros por classe.

12. Roteiro curto para a apresentação

“Nosso trabalho propõe um assistente conversacional para consultas operacionais em um centro de distribuição. O problema é que embarques, agendamentos, pátio, notas fiscais e ocorrências normalmente aparecem em fontes distintas, o que torna consultas simples lentas.

Para demonstrar a ideia sem expor informações reais, criamos a empresa fictícia FBS Distribuição e bancos SQLite com dados sintéticos. A pessoa escreve pelo Telegram. Primeiro, regras e expressões regulares tratam intenções óbvias e identificadores; depois, se necessário, um modelo TF-IDF classifica o texto entre dez intenções. A confiança abaixo de 0,55 leva a um fallback.

Comparamos Multinomial Naive Bayes, LinearSVC calibrado e Random Forest com validação cruzada estratificada em cinco folds, usando F1-macro. O pipeline de LinearSVC calibrado foi escolhido para o bot. No OOD provisório, a estratégia híbrida teve 0,767 de acurácia, contra 0,695 do ML-only, com latência local média reportada de 2,198 milissegundos.

Esses resultados demonstram a viabilidade do protótipo, mas não são uma validação de produção: o teste OOD ainda é sintético e pequeno, e o controle de acesso e o cálculo de estadia precisam de validação adicional. Como próximos passos, vamos coletar exemplos independentes, fortalecer os testes e corrigir essas limitações.”

13. Perguntas que podem surgir

Por que não usar um LLM?

O escopo prioriza uma solução local, leve e previsível para classificar intenções conhecidas. Isso reduz dependência de inferência externa e permite regras explícitas. Um LLM pode ser comparado em trabalho futuro, mas não é necessário para o protótipo.

Por que usar F1-macro?

Porque trata cada intenção igualmente. A acurácia isolada poderia esconder desempenho baixo em classes menos frequentes ou mais difíceis.

O que significa calibrar o LinearSVC?

O LinearSVC fornece escores de margem. CalibratedClassifierCV ajusta uma transformação sigmoid para produzir probabilidades aproximadas, que podem ser comparadas ao limiar de 0,55. Elas não são certezas.

O sistema entende qualquer pergunta?

Não. Ele foi treinado para dez classes específicas. A classe fora de escopo e o fallback tentam evitar uma resposta operacional para perguntas não suportadas.

Os dados são reais?

Não. Os registros são sintéticos, produzidos para a demonstração.

O bot está pronto para uma empresa?

Não. É um protótipo acadêmico. Antes de uso real, é preciso corrigir e validar autorização em cada consulta, segurança, auditoria, regras operacionais, cálculo de estadia, infraestrutura e tratamento de dados pessoais.

O resultado 0,767 significa que o bot terá 76,7% de acerto em produção?

Não. É a acurácia observada no conjunto OOD provisório atual. Como o conjunto é sintético e pequeno, não representa necessariamente conversas reais.

14. Vocabulário essencial

- Intenção: objetivo da mensagem, por exemplo consultar uma NF.
- Entidade: dado citado na mensagem e usado para consulta, por exemplo EMB000010.
- Corpus: coleção de mensagens com seus rótulos.
- TF-IDF: método para converter texto em características numéricas ponderadas.
- N-grama: sequência de palavras; bigrama é uma sequência de duas palavras.
- Calibração: transformação dos escores do modelo em probabilidades aproximadas.
- Fallback: resposta segura quando a mensagem está fora do escopo ou a confiança é baixa.
- Validação cruzada: repetição do treino/validação em diferentes partições dos dados.
- OOD: dados de teste que diferem da distribuição de desenvolvimento.

- Pipeline: sequência reproduzível de pré-processamento, vetorização e classificação.
- SQLite: banco relacional armazenado em arquivo, adequado para a demonstração local.

15. Onde encontrar os principais arquivos

- src/fbs_bot/engine.py: fluxo que coordena interpretação, acesso, consulta e resposta.
- src/fbs_bot/nlp/rules.py: regras de palavras-chave.
- src/fbs_bot/nlp/entities.py: padrões de embarque, placa, NF e correspondência de transportadora.
- src/fbs_bot/nlp/classifier.py: pipeline TF-IDF e carregamento/predição do modelo.
- src/fbs_bot/services/: consultas SQLite, estadia, autorização, auditoria e formatação.
- scripts/seed_db.py: geração dos bancos sintéticos.
- scripts/generate_corpus.py: geração de corpus de desenvolvimento e OOD.
- scripts/train.py: treinamento do modelo usado no bot.
- experiments/run_experiments.py: GridSearchCV e comparação dos modelos.
- experiments/report.md: resultados detalhados do experimento.
- tests/test_core.py: testes automatizados atuais.